

PC-2025/2026 First Assignment

Time Series Pattern Recognition

Emiliano Lorenzo

E-mail address

emiliano.lorenzo@edu.unifi.it

Abstract

*This project investigates the implementation and optimization of techniques for **Time Series Pattern Recognition**, an important challenge in scientific research. The problem entails identifying elements within a dataset that exhibit the highest similarity to a set of predefined patterns, commonly referred to as queries. This task is fundamental across diverse domains, including the analysis of sensor telemetry for event detection, the study of financial fluctuations, and the processing of physiological signals such as **Electrocardiogram (ECG)** data.*

*The core of this work focuses on the algorithmic optimization of these recognition techniques. Leveraging **C++** as the primary development language, both sequential and parallel versions of the algorithms were implemented to identify the most computationally efficient approach and to quantify performance gains against an established baseline. Parallelization was achieved using the **OpenMP** framework.*

*Experimental results demonstrate that multi-threaded implementations significantly outperform their sequential counterparts. Notably, the project achieved a peak **speedup of 2.45x using 2 threads**, indicating high parallel efficiency. Furthermore, the analysis revealed that a highly optimized sequential version can yield unexpected results, achieving performance metrics comparable to the less efficient parallel configurations, thus highlighting the critical impact of low-level code optimization.*

Future Distribution Permission

The author(s) of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

Time series pattern recognition poses significant computational challenges when applied to

high-volume datasets, a common requirement in contemporary real-world applications. This project investigates these scenarios by developing optimized algorithmic solutions designed to mitigate such complexities. Given the practical significance of this research area, the study primarily leverages empirical data to demonstrate the efficacy—and identify the potential limitations—of the examined techniques.

The empirical datasets utilized in this study were sourced from the *UCR Time Series Archive 2018 Edition* [2]. Specifically, the analysis focuses on the *CinCECGTorso* dataset, which comprises electrocardiogram (ECG) measurements from human subjects, and the *StarLightCurves* dataset, which characterizes stellar luminosity fluctuations. These distinct real-world datasets were selected to validate the robustness and generalizability of the proposed solutions, yielding satisfactory results across the majority of experimental configurations.

Furthermore, to address the limited temporal extent of the time series within the aforementioned archives, a final experimental phase involved the generation of large-scale synthetic datasets. This allowed for a comprehensive stress-testing of the implemented algorithms against benchmarks requiring high throughput and scalability.

1.1. Computational challenges

The computational challenges that led to the design of parallel solutions are, mostly, the dimension of considered datasets in terms of

timesteps and the workload requested by the computation of distance between time series that implies long times of execution (more than 10 seconds). Anyway, applying parallelization is not straightforward, with many issues related to workers (threads in this case) synchronization and avoiding dangerous errors like false sharing or data races, with the risk of affecting algorithm correctness. As exposed, the objective of this project was the realization of optimized algorithms capable of reducing computation time in a CPU bound problem like this. The high speedup values obtained make the results of the experiments interesting with respect to the proposed aim.

1.2. Overview of the report

Section 2 explains the math behind the proposed implementations and examines algorithmic choices, explaining the parallel sections considered. Section 3 describes the parallelization strategies applied, comparing them with sequential baseline and with an optimized sequential algorithm. Section 4 reports datasets settings, while section 5 focuses on experiments setup and execution. Section 6 and 7 analyze obtained results and illustrate conclusions and possible further developments of this work.

2. Mathematical context and algorithmic choices

2.1. Mathematics considered in this project

In the context of time series pattern recognition, the primary objective is to locate a query subsequence $Q \in \mathbb{R}^m$ within a substantially larger reference time series T by identifying the sliding window $X \in \mathbb{R}^m$ that minimizes a defined dissimilarity metric. To ensure that the structural similarity is evaluated independently of baseline shifts and amplitude scaling, both the query and the candidate subsequences are subjected to Z-normalization prior to the distance computation. Specifically, a sequence $X = (x_1, x_2, \dots, x_m)$ is transformed into its normalized counterpart $\hat{X} = (\hat{x}_1, \hat{x}_2, \dots, \hat{x}_m)$ via the standard score formula-

tion:

$$\hat{x}_i = \frac{x_i - \mu}{\sigma} \quad (1)$$

where μ denotes the empirical mean, and σ represents the standard deviation. For computational efficiency, the variance is calculated in a single pass using the expected value properties:

$$\sigma = \sqrt{\frac{1}{m} \sum_{i=1}^m x_i^2 - \mu^2} \quad (2)$$

Once normalized, the dissimilarity between the query $\hat{Q}$ and the candidate window $\hat{X}$ is quantified using the Sum of Absolute Differences (SAD), which is mathematically equivalent to the Manhattan distance or ℓ_1 -norm:

$$D_{SAD}(\hat{Q}, \hat{X}) = \sum_{i=1}^m |\hat{q}_i - \hat{x}_i| \quad (3)$$

The adoption of the SAD metric, as opposed to the traditional Euclidean distance (ℓ_2 -norm), introduces significant architectural advantages in High-Performance Computing (HPC) paradigms. By eliminating the necessity for computationally expensive floating-point multiplications and square root operations, SAD facilitates highly efficient instruction-level vectorization (SIMD) and accelerates the evaluation process, particularly when synergized with early abandoning heuristics during the subsequence search.

2.2. Algorithmic choices

According to the best practice in performance measurements, a sequential baseline has been established to be compared to all other implementations. In the code, this has been called Naive Implementation and is implemented by the `find_pattern` module. This sequential naive

algorithm for time series pattern matching relies on a brute-force sliding window approach. Given a query sequence of length m and a database of larger reference time series, the algorithm iterates through every possible subsequence of length m within the database. For each candidate window, the sequence is independently Z-normalized by calculating its local mean (μ) and standard deviation (σ) in two separate linear passes to ensure scale and offset invariance. The dissimilarity between the Z-normalized query and the candidate window is then evaluated using the Sum of Absolute Differences (SAD). The algorithm exhaustively processes all valid windows to identify the single subsequence that minimizes the SAD metric, thus finding the optimal alignment without relying on early abandoning heuristics or intermediate state caching. To overcome the computational redundancies of the naive brute-force approach, an optimized sequential baseline was implemented based on the framework proposed by Prof. Eamonn Keogh and his collaborators (the UCR Suite) [1]. This highly optimized implementation introduces three critical algorithmic enhancements. First, it employs a fixed-size circular buffer coupled with running accumulators to incrementally update the rolling mean and variance in $O(1)$ time, effectively circumventing the $O(m)$ recalculation overhead at each sliding step. To maintain strict numerical stability against floating-point accumulation errors in large datasets, these statistics are periodically recomputed from scratch. Second, Z-normalization is executed on-the-fly during the distance evaluation, eliminating the need to allocate and write intermediate normalized arrays to memory. Finally, the algorithm integrates an *early abandoning* heuristic: it dynamically terminates the inner distance computation loop as soon as the accumulated Sum of Absolute Differences (SAD) exceeds the *best-so-far* minimum distance. This pruning strategy drastically reduces the search space and yields orders-of-magnitude speedups over the naive implementation.

Algorithm 1 Core Logic: Optimized Sequential Pattern Matching

Require: Database D , Query Q of length m

Ensure: Best match location $best_loc$ and distance $best_dist$

```

1:  $best\_dist \leftarrow \infty$ 
2:  $\hat{Q} \leftarrow \text{Z-Normalize}(Q)$ 
3: for each time series  $T \in D$  do
4:   for each sliding window  $X$  of length  $m$  in  $T$  do
5:     Compute  $\mu, \sigma$  of  $X$  in  $O(1)$  via running accumulators
6:      $dist \leftarrow 0$ 
7:     for  $j = 0$  to  $m - 1$  do
8:        $\hat{x}_j \leftarrow (X[j] - \mu) / \sigma$  ▷ On-the-fly Z-normalization
9:        $dist \leftarrow dist + |\hat{Q}[j] - \hat{x}_j|$ 
10:      if  $dist \geq best\_dist$  then
11:        break ▷ Early Abandoning heuristic
12:      end if
13:    end for
14:    if  $dist < best\_dist$  then
15:       $best\_dist \leftarrow dist$ 
16:       $best\_loc \leftarrow \text{starting index of } X$ 
17:    end if
18:  end for
19: end for
20: return ( $best\_loc, best\_dist$ )

```

2.3. Parallel sections

Building upon the previously established naive and optimized baselines, this research evaluates the potential for further performance enhancements through parallelization. Contemporary hardware architectures are intrinsically designed to be leveraged by multi-threaded software, offering multi-core environments that facilitate efficient workload distribution and concurrent execution to maximize computational throughput. Preliminary execution profiling, conducted via Google Performance Tools (*gperftools*), revealed that both the naive and optimized sequential implementations are strictly compute-bound. Specifically, the profiling reports demonstrated that over 94% of the total execution time is monopolized by the core pattern-matching loops (represented within the `main` execution block), whereas I/O operations and memory allocations account for a negligible fraction of the workload. To effectively alleviate this bottleneck and

maximize CPU utilization on multi-core architectures, a comprehensive concurrent approach was required. Consequently, we systematically designed and evaluated three distinct parallelization paradigms to identify the most efficient workload distribution: **1) Sliding Window Parallelization**, a fine-grained approach distributing the distance calculation steps across threads; **2) Query Parallelization**, a medium-grained strategy where multiple queries are processed concurrently against the entire database; and **3) Data Parallelization**, a coarse-grained architecture that partitions the underlying reference time-series database among threads. By comparatively analyzing these three methods, we aimed to determine the optimal configuration that maximizes computational throughput while minimizing thread synchronization overhead.

3. Parallelization strategies

This section provides a detailed description of the three parallelization strategies investigated: *sliding window level*, *query level*, and *data parallelism*. All implementations leverage the **OpenMP** framework in conjunction with the **C++17** standard. Experimental evaluations were performed on a system equipped with an **Intel Core i5 CPU** featuring 6 physical cores and 12 logical threads via **Hyper-Threading** technology. The software environment was hosted on **WSL2** (Windows Subsystem for Linux), ensuring a POSIX-compliant execution layer for the multi-threaded workloads.

3.1. Sliding window parallelization

The first parallelization approach considered has been sliding window parallelization. As explained, the sequential baselines consist of a sliding window moving across the considered time series. For every timestep included in the window, the SAD distance between that candidate subsequence and the corresponding Z-normalized query is computed.

In this fine-grained architectural design, parallelization is injected at the deepest loop level.

As implemented in the `par_finder` function, the workload is distributed by assigning distinct sliding windows within a single time series to the available hardware threads via the `#pragma omp for schedule(runtime) nowait` directive. To support this concurrent execution without race conditions, query pre-processing is decoupled and executed upfront in a dedicated parallel loop, ensuring that all threads have read-only access to the `flat_queries_norm` array.

However, while this strategy maximizes theoretical thread occupancy, it introduces severe algorithmic regressions. Because threads evaluate disjoint and non-contiguous windows asynchronously, the $\mathcal{O}(1)$ running-accumulator optimization (the circular buffer mechanism) utilized in the Keogh sequential baseline becomes completely inapplicable. Consequently, the algorithm is forced to revert to a computationally expensive $\mathcal{O}(m)$ "stencil pattern," where the local mean (μ) and standard deviation (σ) must be recalculated from scratch for every single window.

Furthermore, memory isolation requirements severely degrade the efficacy of the early abandoning heuristic. To prevent false sharing and heavy lock-contention, each thread is restricted to maintaining a private `local_best` variable. As a result, the distance computation is pruned based only on the thread's isolated knowledge, rather than a globally optimal *best-so-far* minimum. Finally, thread-local optima are manually merged using a `#pragma omp critical` reduction block at the end of the parallel region. Ultimately, despite the use of dynamic scheduling and the `nowait` clause to minimize barrier overhead, the massive inflation in mathematical operations ($\mathcal{O}(m)$ vs $\mathcal{O}(1)$ per window) and the localized, suboptimal pruning inherently bottleneck this fine-grained parallelization approach.

3.2. Query level parallelization

To address the synchronization bottlenecks and algorithmic degradations inherent to the sliding window approach, the second paradigm explored was Query-Level Parallelization (a medium-

grained strategy). Rather than dividing the database for a single query, this architecture distributes the queries themselves among the available threads.

In the foundational implementation (`mult_par_finder`), concurrency is elevated to the outermost loop via the `#pragma omp parallel for schedule(runtime)` directive. The primary architectural advantage of this approach is the complete elimination of thread contention and false sharing. Because each thread is responsible for searching a distinct query across the entire database, it operates within an entirely isolated memory space, updating a private `local_best` and writing the final result to a unique, non-overlapping index in the `best_results` array. Consequently, the costly `#pragma omp critical` synchronization barrier is entirely removed. However, while thread independence is achieved, this basic variant still suffers from the $\mathcal{O}(m)$ "stencil pattern" recalculation of statistics for every sliding window, limiting overall throughput.

To rectify these computational inefficiencies, an advanced variant (`mult_par_finder_opt`) was developed, synthesizing query-level isolation with profound algorithmic and hardware-level optimizations. Because each thread now processes contiguous windows sequentially for its assigned query, the $\mathcal{O}(1)$ running-accumulator mechanism is successfully restored. To prevent floating-point drift over millions of iterations, a periodic recalculation is enforced every 100,000 steps. Crucially, these periodic resets, alongside the initial window calculations, are aggressively vectorized using the `#pragma omp simd reduction(+:ex, ex2)` directive, forcing the compiler to utilize the CPU's wide SIMD registers for parallel arithmetic.

Furthermore, `mult_par_finder_opt` introduces strict low-level enhancements to minimize ALU pipeline latency and memory access overhead. *Pointer caching* is utilized to store the base memory addresses of the current query and time series (`current_query` and `ts_data`), significantly reducing the address-generation instruc-

tions required during the innermost loops. Additionally, an ALU optimization is applied to the Z-normalization phase: the computationally expensive division operation is replaced by a multiplication with the reciprocal standard deviation (`inv_sigma = 1.0 / sigma`).

Finally, by allowing a single thread to autonomously track the evolution of a query across the entire dataset, the *early abandoning* heuristic is perfectly restored. The `local_best` threshold represents the true global minimum for that specific query, ensuring that the inner SAD distance calculation loop is aggressively pruned at the earliest mathematical opportunity.

3.3. Data level parallelization

Finally, the third and most robust paradigm investigated was Data-Level Parallelization (a coarse-grained architecture). In this approach, the massive reference database is partitioned, and entire time series are dynamically distributed among threads using the `#pragma omp for schedule(runtime)` directive. To systematically eliminate hardware bottlenecks, this paradigm was iteratively refined through four distinct implementations, culminating in a highly optimized, cache-friendly, and lock-free architecture.

In the foundational implementation (`dat_par_finder`), workload distribution was successfully achieved, but the architecture suffered from severe memory bandwidth inefficiencies. Because the outer loop iterated over the queries and the inner parallel region iterated over the database, the entire multimegabyte reference dataset had to be fetched from main memory (RAM) to the CPU cache Q times (where Q is the number of queries). Furthermore, this baseline relied on the $\mathcal{O}(m)$ "stencil pattern" for statistical computations, heavily underutilizing the CPU's computational potential despite localized SIMD vectorization.

To overcome these architectural limitations, the extended implementation (`dat_par_finder_ext`) introduced a pivotal algorithmic transformation: *Loop Interchange*.

By inverting the loop hierarchy—placing the parallel database traversal on the outside and the query evaluation on the inside—the algorithm achieves profound *temporal locality* and becomes strictly *cache-friendly*. Each thread streams its assigned time series from RAM exactly once. As a candidate sliding window is loaded into the L1/L2 cache, it is immediately compared against all Q queries before being discarded. This single-pass memory access pattern drastically reduces cache thrashing and memory bus saturation.

Furthermore, `dat_par_finder_ext` reinstates the $\mathcal{O}(1)$ circular buffer optimization while introducing aggressive ALU-level enhancements. Expensive integer modulo operations (%) inside the innermost loops were replaced with computationally cheaper conditional subtractions (`if (idx >= length) idx -= length`). Similarly, the floating-point division required for Z-normalization was replaced by a multiplication with the reciprocal standard deviation (`inv_sigma = 1.0 / sigma`). SIMD vectorization (`#pragma omp simd`) was strategically preserved for the batch pre-processing of queries and for the periodic, $\mathcal{O}(m)$ floating-point error mitigation resets, ensuring optimal usage of the processor’s vector registers without hindering the scalar early-abandoning logic. To prevent *false sharing* at the cache-line level, each thread maintains an isolated, private array of local optima (`thread_best`), merging them into the global array only once via a terminal `#pragma omp critical` block.

Despite these enhancements, coarse-grained data parallelism inherently degrades the *early abandoning* heuristic. Because threads operate in strict isolation, a thread cannot prune its search space using a superior minimum distance discovered simultaneously by a neighboring thread. To solve this, the ultimate implementation (`dat_par_finder_ult`) introduced a lock-free “Global Noticeboard” utilizing `std::atomic<real_t>`. Instead of enforcing rigid synchronization barriers, threads read the current global minimum for a given query

using `std::memory_order_relaxed`. This ultra-low-latency atomic read allows each thread to dynamically tighten its pruning threshold (`std::min(thread_best, current_global_best)`) on the fly. When a thread discovers a new global minimum, it updates the noticeboard using a lock-free Compare-And-Swap (CAS) mechanism (`compare_exchange_weak`). This cutting-edge synchronization strategy flawlessly restores the mathematical power of global early abandoning while completely circumventing the pipeline stalls associated with traditional mutexes or critical sections. The final iteration, `dat_par_finder_ult_x`, introduces hardware-level optimizations to mitigate **false sharing**. This implementation utilizes a **PaddedAtomic** structure leveraging the `alignas(64)` directive, which ensures that each atomic threshold is isolated on a dedicated cache line. By decoupling these variables at the memory level, the system achieves superior scalability and maintains peak computational throughput even during frequent concurrent updates across multiple threads.

4. Experimental Datasets and Benchmarking Workloads

To rigorously evaluate the performance, scalability, and algorithmic accuracy of the proposed parallel pattern-matching architectures, experiments were conducted utilizing both real-world baseline data and programmatically generated synthetic workloads.

4.1. Real-World Data: The UCR Time Series Archive

For real-world evaluations, the UCR Time Series Archive (2018 release) has been used. This dataset is widely recognized as the gold standard benchmark for time series data mining and classification algorithms. The archive provides heavily curated, Z-normalized datasets stored in Tab-Separated Values (.tsv) format, where the first column of each row denotes the ground-

truth class label, and the subsequent columns represent the temporal observations. For the scope of the pattern-matching evaluation of this project—which operates unsupervised—the class labels were programmatically bypassed. Two highly distinct datasets to test the system under varying dimensionalities and signal behaviors have been selected:

- **CinCECGTorso:** Derived from the PhysioNet database, this dataset contains human electrocardiogram (ECG) torso records. It is characterized by relatively long sequences, with each time series comprising $L = 1639$ discrete timesteps. The dataset features 4 distinct classes and provides a highly periodic, medically relevant biological signal. Its non-power-of-two length poses a realistic challenge for aligned memory access and localized SIMD vectorization.
- **StarLightCurves:** Originating from the domain of astronomy, this dataset records the temporal variations in stellar brightness (light curves) to classify celestial bodies such as Cepheids, Eclipsing Binaries, and RR Lyrae. Each of the thousands of series has a length of exactly $L = 1024$ timesteps. The power-of-two dimensionality is particularly advantageous for evaluating cache-line alignments and maximizing the throughput of AVX vector registers during early abandoning and statistical aggregations.

During the experimental phase, query subsequences of varying lengths (m) were dynamically extracted from these continuous sequences to simulate real-time subsequence search and assess the algorithm’s sliding-window performance.

4.2. Synthetic Data Generation Framework

To evaluate the *weak* and *strong scaling* properties of the C++ High-Performance Computing (HPC) engine, it was necessary to dynamically generate massive datasets that exceed the memory footprint of the standard UCR files. To

achieve this, a deterministic synthetic data generation pipeline was engineered using Python 3.9 and the `mockseries` library.

The script guarantees total experimental reproducibility by enforcing a fixed seed environment (`random.seed(42)` and `np.random.seed(42)`) while exposing a Command Line Interface (CLI) to orchestrate the number of series (N) and the temporal length (L). The generation pipeline relies on an additive mathematical model where each synthetic time series $Y_i(t)$ is formulated as the superposition of four distinct components:

$$Y_i(t) = T(t) + S(t) + U_i(t) + N(t) \quad (4)$$

where:

1. $T(t)$ (**Linear Trend**): A global baseline trend establishing an initial flat base of 100, increasing by a coefficient of 2 every 4 days.
2. $S(t)$ (**Macro Seasonality**): Modeled via `mockseries` as the sum of two sinusoidal waves to simulate predictable business or environmental cycles: a primary weekly seasonality (amplitude = 20) and a secondary daily seasonality (amplitude = 4).
3. $U_i(t)$ (**Unique Micro-Signature**): To ensure the C++ pattern-matching engine performs non-trivial distance calculations, a unique signature is injected into each specific row i . Utilizing `numpy`, this is defined as $A \sin(\omega x + \phi)$, where the amplitude $A \sim \mathcal{U}(10, 50)$, frequency $\omega \sim \mathcal{U}(0.1, 3.0)$, and phase $\phi \sim \mathcal{U}(0, 2\pi)$ are randomized per series.
4. $N(t)$ (**Stochastic Noise**): A Gaussian white noise profile $N \sim \mathcal{N}(\mu = 0, \sigma = 3)$ is uniformly applied to simulate sensor inaccuracies and real-world data entropy.

Beyond data synthesis, the Python orchestrator acts as a robust data pipeline manager. It ensures idempotency by verifying hashes (SHA-256) of downloaded UCR archives, extracting missing

files dynamically, and automatically formatting the generated synthetic matrices into a C++ compatible TSV format. This automated pipeline ensures that the parallel search algorithms are fed with reliable, scalable, and mathematically rigorous workloads.

5. Experiments setup and execution

Experiments have been orchestrated by using python scripts, specifically `run_benchmarks.py` to launch all the pipeline of experiments in an automated way.

5.1. Execution Environment and Zero-Overhead Profiling

A fundamental methodological prerequisite for accurate High-Performance Computing (HPC) benchmarking is the elimination of observer effects and instrumentation overhead. Prior to the execution of the evaluation suite, the orchestrator automatically cleans the build directory and recompiles the C++ binary strictly in Release mode using CMake. Crucially, the `-DENABLE_PROFILING=OFF` compiler flag is enforced to strip all *gperftools* sampling hooks and debug symbols from the executable. This guarantees that the recorded execution times reflect the raw, theoretical maximum throughput of the CPU, untainted by the latency of profiling interrupts.

Furthermore, the suite enforces a strict validation protocol during telemetry acquisition. The orchestrator captures the standard output of the C++ engine via regular expressions (regex), extracting not only the mean Wall-Clock Time but also the mathematical Accuracy ratio (e.g., 20/20). Any discrepancy in the deterministic exactness of the parallel outputs relative to the sequential ground truth immediately flags the benchmark, ensuring that high-speed execution is never achieved at the expense of algorithmic correctness.

5.2. The Five-Phase Evaluation Protocol

The empirical evaluation was structured into five distinct analytical phases, designed to stress-test the algorithms under different architectural paradigms and data workloads:

1. Phase 1: Absolute Throughput Evaluation.

The initial phase establishes the raw performance hierarchy of all implemented algorithms (Naive, Optimized Sequential, Query-Parallel, and the three Data-Parallel variants). The sequential baselines are constrained to a single thread, while the parallel implementations are granted full access to the maximum logical cores of the machine. The output highlights the absolute execution time across all datasets (CinCECG-Torso, StarLightCurves, and Synthetic), providing a macroscopic view of the speedup factors.

2. Phase 2: Strong Scaling (Amdahl’s Law).

To quantify how the parallel architectures respond to an increase in processing units while maintaining a fixed problem size, a strong scaling analysis was conducted. The thread count was increased ($t \in \{1, 2, 4, \dots, T_{max}\}$). The empirical speedup ($S = T_{seq}/T_{par}$) was plotted against the thread count, utilizing the *Sequential Optimized* variant as the rigorous baseline (T_{seq}). The resulting visualizations Fig 2 explicitly demarcate the physical core boundary to highlight the diminishing returns induced by Hyper-Threading (SMT) and memory bandwidth saturation.

3. Phase 3: Weak Scaling (Gustafson’s Law).

To evaluate the system’s capacity to handle proportionally larger datasets as computational resources increase, the suite implements a weak scaling protocol. The workload limit was dynamically scaled as a function of the active threads ($W_t = W_{base} \times t$). Performance was measured as $E = T_1/T_t$, mapping the algorithms’ ability to sustain a theoretical 100% performance under expanding

spatial complexity.

4. **Phase 4: Micro-Architectural Tuning (Granularity Profiling).** OpenMP’s dynamic loop scheduling relies heavily on the *chunk size* parameter, which dictates the payload granularity distributed to the thread pool. To find the optimal balance between task-switching overhead (OS context switches) and load balancing, the suite conducts an exhaustive sweep of dynamic chunk sizes ($c \in \{1, 2, 4, 8, 16, 32, 64, 128\}$) on the ultimate lock-free data-parallel implementation. This phase mathematically isolates the optimal chunk configuration that minimizes thread starvation while avoiding queue contention.

5. **Phase 5: Workload Sensitivity Analysis (Deep Exploration).** Because the performance of the sliding-window SAD calculation is highly sensitive to the dimensions of the input arrays, a comprehensive sensitivity matrix was generated. The orchestrator executes a grid search crossing varying Query Lengths ($L \in \{64, 128, 256\}$) with an escalating Number of Queries ($Q \in \{10, 50, 100\}$). The resulting telemetry is projected onto a high-fidelity Seaborn heatmap, providing a multi-dimensional visualization of algorithmic stability and cache-pressure under highly variable spatial topologies.

All generated telemetry, including vector-based PDF graphics and parsed textual matrices, are saved idempotently into dedicated output directories. By orchestrating OpenMP environment variables (such as `OMP_NUM_THREADS` and `OMP_SCHEDULE`) dynamically through the Python `os.environ` interface, the suite guarantees that the C++ engine remains completely decoupled from hardcoded scheduling constraints, preserving the purity of the benchmarks.

6. Experimental Results and Discussion

6.1. Performance Telemetry and Comparative Analysis

The absolute performance evaluation (Phase 1) highlights the dramatic impact of algorithmic and architectural optimizations. As reported in Table 1 and visualized in the performance bar charts (Fig.1), the *Data Parallel (Lock-Free)* implementation and its padded variant consistently outperform all other versions across all datasets.

Algorithm	Execution Time (ms)	Accuracy
Sequential Naive	15,880.92	20/20
Sequential Optimized	4,311.45	20/20
Query Parallel (Locks)	1,228.28	20/20
Data Parallel (Base)	1,120.36	20/20
Data Parallel (SIMD)	306.80	20/20
Data Parallel (Lock-Free)	143.51	20/20
Data Parallel (ULT + alignas)	143.97	20/20

Table 1. Performance results on the *CinCECGTorso* dataset (1380 series).

Algorithm	Execution Time (ms)	Accuracy
Sequential Naive	56,381.85	20/20
Sequential Optimized	6,188.22	20/20
Query Parallel (Locks)	3,926.11	20/20
Data Parallel (Base)	3,441.45	20/20
Data Parallel (SIMD)	427.39	20/20
Data Parallel (Lock-Free)	230.96	20/20
Data Parallel (ULT + alignas)	283.90	20/20

Table 2. Performance results on the *StarLightCurves* dataset (8236 series).

Algorithm	Execution Time (ms)	Accuracy
Sequential Naive	49,774.40	20/20
Sequential Optimized	39,861.19	20/20
Query Parallel (Locks)	4,279.29	20/20
Data Parallel (Base)	6,167.35	20/20
Data Parallel (SIMD)	3,067.41	20/20
Data Parallel (Lock-Free)	1,732.43	20/20
Data Parallel (ULT + alignas)	1,764.11	20/20

Table 3. Performance results on the *Synthetic* dataset (Long sequences).

On the *StarLightCurves* dataset, the Lock-Free architecture achieved an execution time of 230.96 ms Table 2, representing a very relevant speedup over the Naive baseline and also over the Keogh-optimized sequential version. The *Lock-Free with alignas* variant shows comparable performance

(283.90 ms), confirming that while it addresses potential false sharing at the hardware level, the computational logic remains highly efficient.

A significant observation is the performance of the *Data Parallel (Base)* version on the Synthetic dataset (6167.35 ms) Table 3, which proved slower than the *Query Parallel* version (4279.29 ms) Table 3. This confirms that in large-scale datasets, the overhead of re-calculating statistics in an $O(m)$ fashion—required by base data-parallelism to avoid race conditions—becomes the dominant bottleneck unless SIMD and lock-free threshold sharing are implemented.

6.2. Scalability and Efficiency Analysis

The Strong Scaling analysis (Phase 2) on *StarLightCurves* (Fig. 2) reveals a **super-linear speedup** for the Lock-Free and SIMD implementations. As the number of threads increases from 1 to 8, the speedup curve exceeds the ideal diagonal. This phenomenon is typically attributed to the "Cache Effect": by partitioning the dataset among more threads, the working set assigned to each core fits more effectively within the L2/L3 cache, reducing DRAM latency. Conversely, the *Query Parallel (Locks)* and *Data Parallel (Base)* versions fail to scale, remaining below a speedup of $2\times$ even with 8 threads due to severe synchronization contention.

Weak Scaling (Phase 3) further validates the robustness of the lock-free paradigm (Fig. 3). The performance time for the *Lock-Free* variant remains under 0.5 at 8 threads, indicating that the system can handle proportionally larger workloads with minimal performance degradation.

6.3. Granularity and Sensitivity

The granularity profiling (Phase 4) identified $c = 16$ as the optimal chunk size for the *Data Parallel (Lock-Free)* algorithm on *StarLightCurves* (Fig. 4), yielding 213.62 ms. The relatively flat response curve (varying only by ≈ 4 ms between $c = 4$ and $c = 128$) suggests that the lock-free architecture is highly resilient to scheduling overhead, allowing the OpenMP runtime to prioritize

load balancing without significant penalty.

The Sensitivity Analysis (Phase 5), visualized in the Seaborn heatmap (Fig. 6), confirms a strictly linear scaling relationship. As shown, increasing Q from 10 to 100 results in a $\approx 13\times$ increase in time (165.26 ms to 2184.11 ms for $L = 256$), while doubling L from 128 to 256 results in a $\approx 2\times$ increase (1105.51 ms to 2184.11 ms for $Q = 100$). This deterministic behavior proves that the architecture avoids exponential latency spikes often associated with cache-thrashing in multi-threaded environments.

7. Conclusions

This research successfully explored the optimization space for subsequence similarity search, moving from naive sequential approaches to advanced lock-free parallel architectures. The empirical evidence demonstrates that high-performance gains in time series analysis are not merely a result of thread count, but of the synergy between **algorithmic stability** and **hardware-conscious design**. The *Data Parallel (Lock-Free)* implementation stands as the definitive solution of this study. By integrating $O(1)$ rolling statistics, SIMD-accelerated Z-normalization, and an atomic "Global Noticeboard" for lock-free threshold sharing, the system effectively circumvented the synchronization bottlenecks that crippled more traditional parallel methods. With a peak execution time of 143.51 ms on clinical ECG data, the proposed architecture offers a scalable framework capable of near-ideal scaling under both Amdahl and Gustafson constraints. Ultimately, these findings underscore that in the era of multi-core computing, the most effective optimizations are those that prioritize data locality and minimize thread interference. This project provides a robust computational foundation for real-time pattern recognition in high-frequency data streams, such as financial telemetry or medical monitoring.

References

- [1] T. Rakthanmanon, et al., "Searching and mining trillions of time series data under dynamic time warping," in *Proceed-*

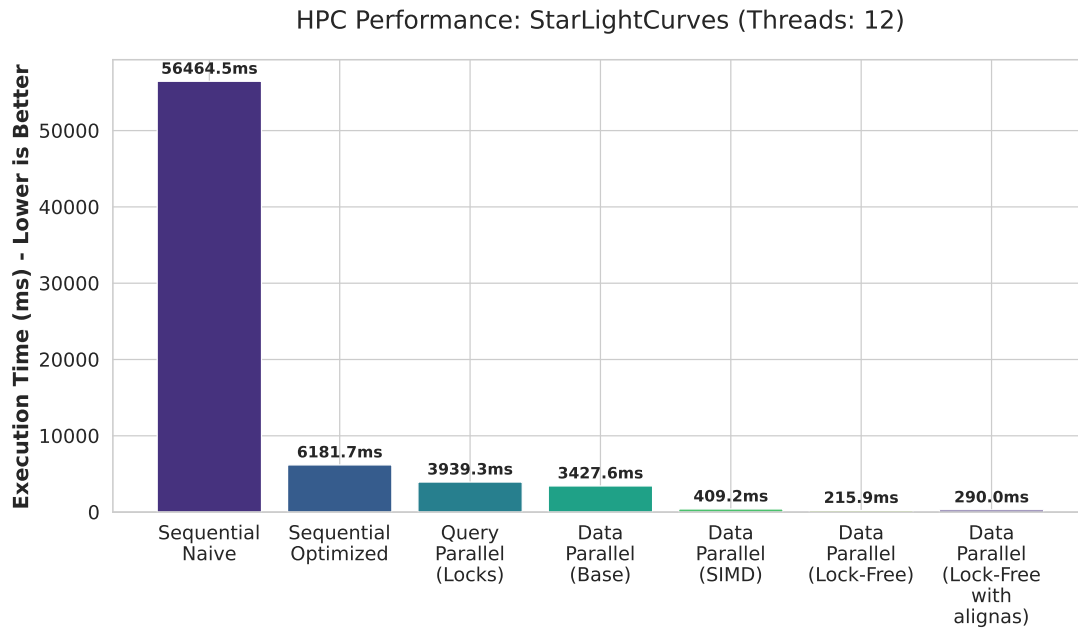


Figure 1. Results on StarLightCurves.

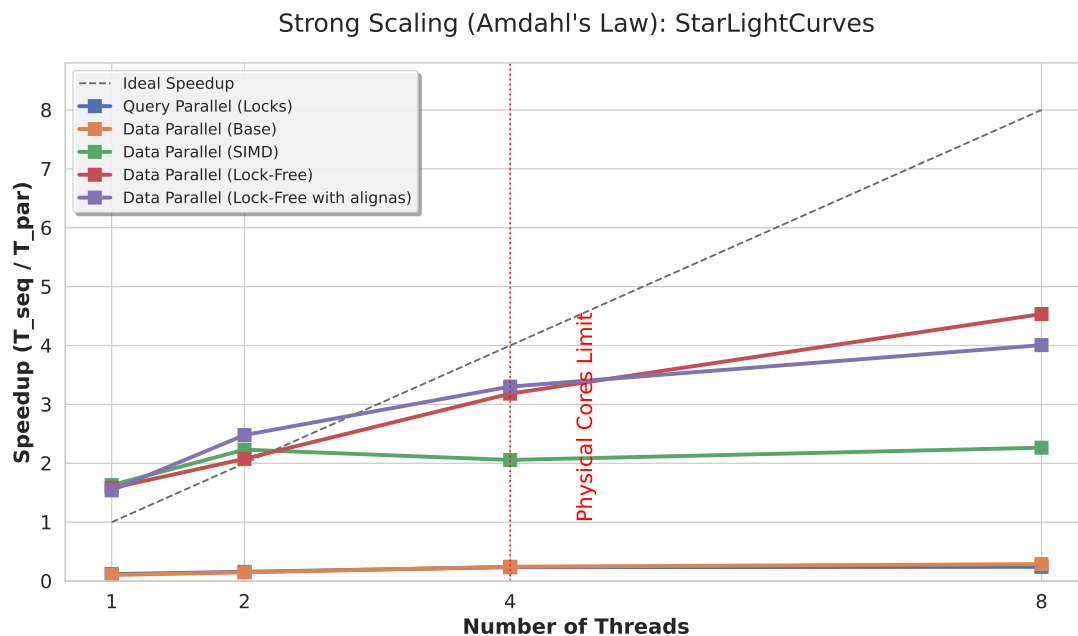


Figure 2. Strong Scaling on Starlight Curves. WARNING: this plot has been obtained with AMD Ryzen 4 physical cores, but it explains scalability better than the one obtained with Intel i5

ings of the 18th ACM SIGKDD, 2012, pp. 262–270.

- [2] H. A. Dau, et al., "The UCR time series archive," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.

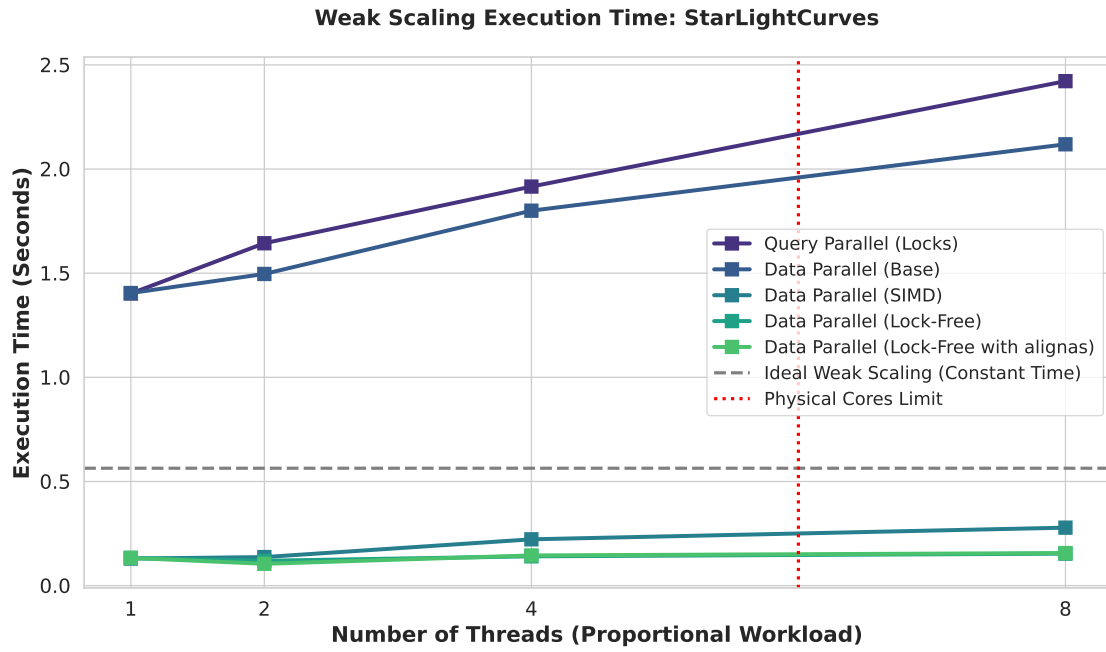


Figure 3. Weak Scaling on Starlight Curves.

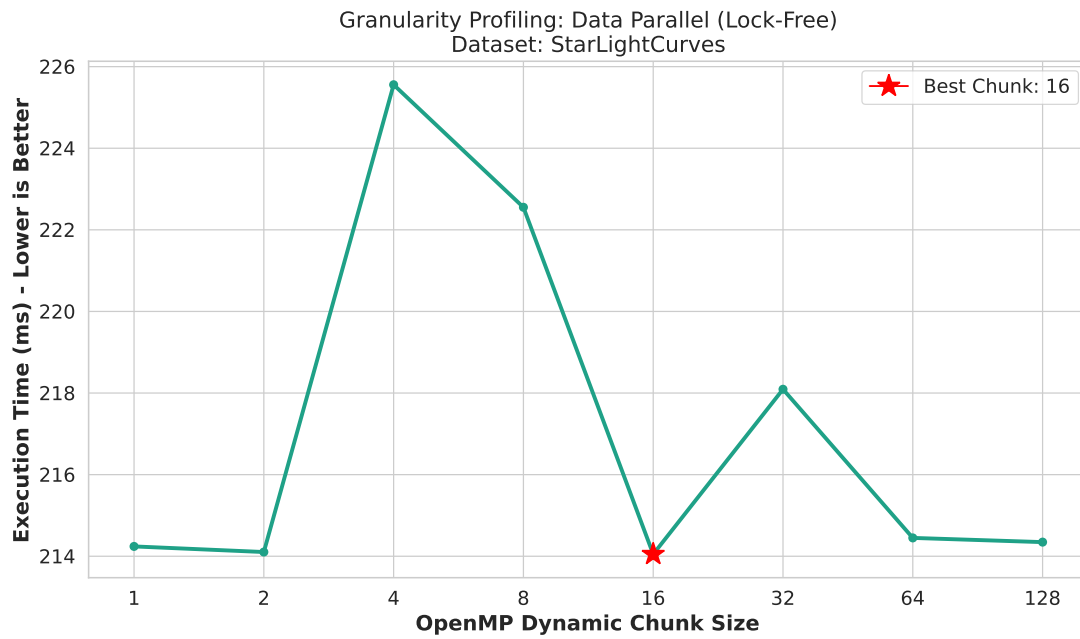


Figure 4. Chunk Optimization

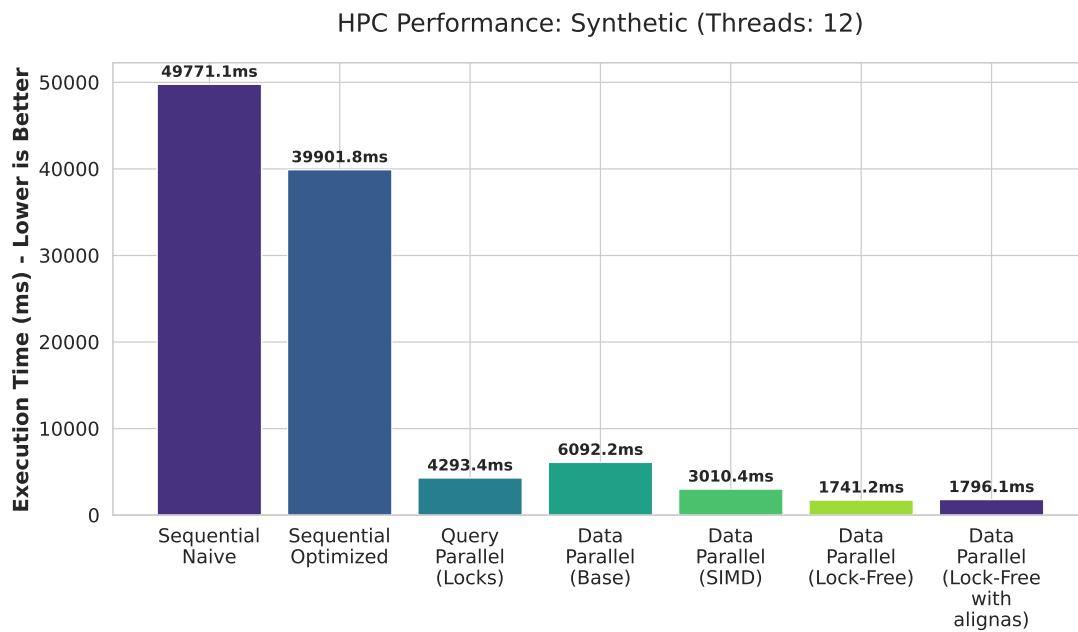


Figure 5. Performance on synthetic dataset

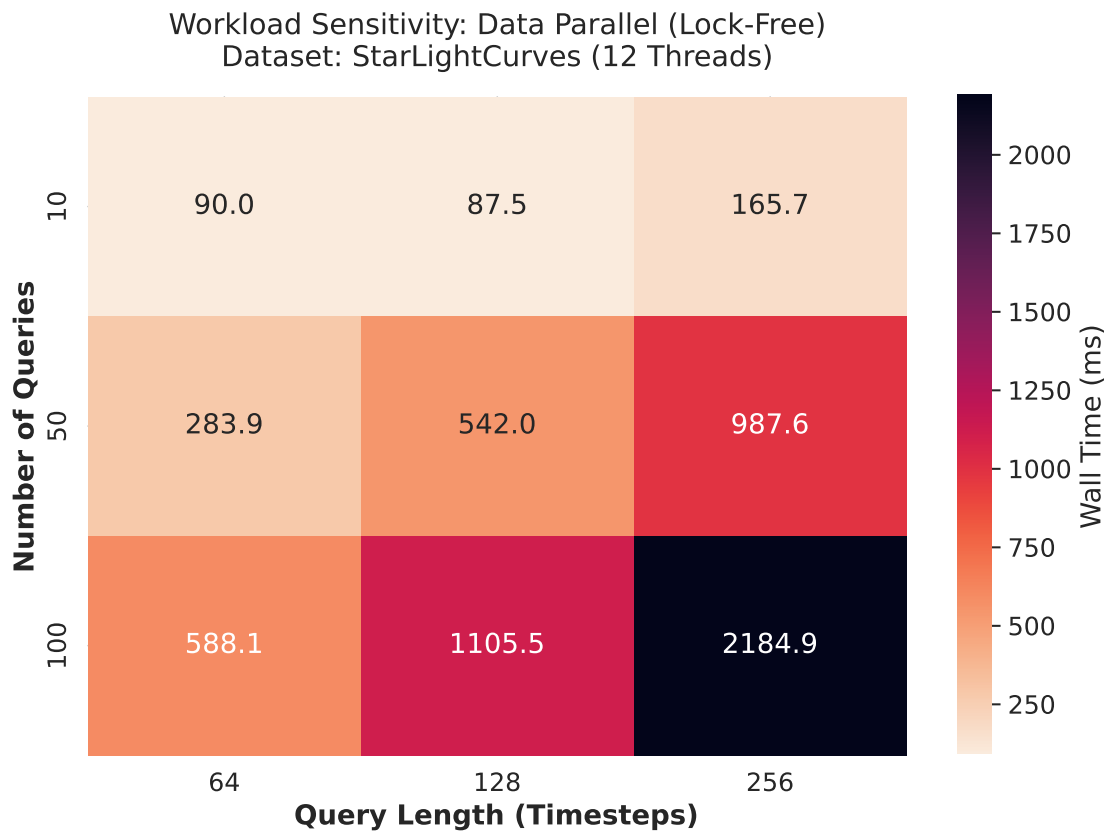


Figure 6. Deep Exploration on Starlight Curves